



Fase 9 — Dashboard com Streamlit

Até agora tudo que o projeto produzia eram arquivos — CSVs de relatório, Markdown, pickle do modelo. Útil pra quem mexe com dados, mas ruim pra mostrar pro cliente ou usar no dia a dia sem abrir terminal.

Então nessa fase eu construí um dashboard com Streamlit que junta tudo em um lugar só: os achados das regras de auditoria, os resultados do ML, tudo numa interface navegável.

O que tem no dashboard

6 seções, cada uma com tabelas e gráficos:

- **Visão Geral** — cards com resumo de tudo, gráfico de pizza e barra com a distribuição dos achados, e sugestão de prioridade de resolução
- **Divergências de Pagamento** — gráfico por empresa mostrando onde estão as maiores diferenças, mais tabela detalhada
- **Conciliações** — conciliações que não fecharam, agrupadas por empresa
- **Movimentações sem Conciliação** — as 388 transações no extrato que ainda não têm título associado, com gráfico de evolução no tempo
- **Documentos Fiscais** — duas abas: documentos com problema (cancelados/inutilizados) e duplicados
- **Anomalias ML** — o que o Isolation Forest sinalizou, com gráfico de score por conta e scatter de diferença vs. atraso

Por que Streamlit?

Três motivos:

1. **É Python puro** — sem precisar aprender HTML/CSS/JS ou mexer em React. Escreve o que faria num script e vira interface.
2. **Ótimo pra portfólio** — qualquer pessoa consegue rodar com um comando só.
3. **Suficiente pro propósito** — não é pra escalar pra 10.000 usuários. É pra auditor usar localmente antes de fechar o mês.

Como rodar

```
# instala a dependência (se não tiver)
pip install streamlit plotly

# sobe o dashboard
streamlit run src/dashboard/app.py
```

Abre em `http://localhost:8501` automaticamente.

Estrutura do código

O `app.py` é um arquivo só, organizado em seções por página:

```
app.py
├─ config da página
├─ helpers (ler csv, formatar valores, montar card)
├─ carrega os dados (com cache do Streamlit)
├─ sidebar com navegação
└─ uma seção por página (if/elif)
```

O Streamlit tem um cache embutido (`@st.cache_data`) que evita ler os CSVs do disco toda vez que o usuário muda de aba — importante quando o volume de movimentações for maior.

Escolhas de design

- **Sem banco de dados** — lê direto dos CSVs de `data/processed/` . Se o banco estiver rodando e o usuário rodar as fases anteriores, os arquivos já estão lá.
- **Plotly** pra gráficos — interativo por padrão (zoom, hover, download). Bem melhor que matplotlib num dashboard.
- **Cards simples** na visão geral — valores grandes e rápidos de ler, sem poluição visual.
- **Valores em pt-BR** — `R$ 56.998,59` , não `R$ 56,998.59` .

Próxima fase

- **Fase 10** — Docker e documentação final (empacotar tudo pra rodar com um `docker-compose up`)